

UNIVERSITY OF AZAD JAMMU & KASHMIR CHELLAH
CAMPUS MUZAFFERABAD



DEPARTMENT OF SOFTWARE ENGINEERING

Databases Systems

(Open-Ended-Lab)

Submitted By:

Hasnain Malik
(2024-SE-37)

Submitted To:

Respected “Engr. Muhammad Awais”

Session: 2024-2028

1. Introduction and Problem Scenario

CarGo Rentals is a local vehicle rental company that has relied on spreadsheets to manage customers, vehicles, rentals, and payments. As the business has grown, this approach has led to duplicate data, inconsistent records, and difficulty generating reliable operational reports.

This report presents a relational database solution for CarGo Rentals. It covers the schema design and SQL implementation (Task 1), normalization of the original flat record from 1NF to 3NF (Task 2), four business-driven JOIN queries (Task 3), a consolidated reporting view (Task 4), an automated trigger and stored procedure for rental operations (Tasks 5 & 6), and a performance optimization analysis (Task 7).

2. Task 1 — Database Design and Implementation

The system is modelled with four related tables: Customers, Vehicles, Rentals, and Payments. Splitting the data this way means every customer, vehicle, and rental transaction is stored exactly once, and relationships are expressed through foreign keys instead of by repeating information in every row.

2.1 Design Justification

- customer_id and vehicle_no act as primary keys — customer_id is a surrogate auto-increment key, while vehicle_no (the registration number) is used directly as the vehicle's natural key since it is already unique and human-meaningful.
- rentals links one customer to one vehicle through the customer_id and vehicle_no foreign keys, and payments links back to a single rental through rental_id, keeping payment records isolated for auditing.
- The core business rule — a vehicle cannot be rented while it is already rented — is enforced at the database level through the vehicles.status column together with triggers described in Section 6, not left to the application to check.
- CHECK constraints keep values valid (positive daily rates, non-negative payment amounts, valid status values); UNIQUE constraints stop duplicate phone numbers and duplicate payments per rental; DEFAULT values reduce the data that must be entered manually.

2.2 Table Structures

Customers

Column	Type / Constraint	Description
customer_id	INT, PK, AUTO_INCREMENT	Surrogate primary key
customer_name	VARCHAR(100), NOT NULL	Customer's full name
phone	VARCHAR(15), NOT NULL, UNIQUE	Primary contact number
email	VARCHAR(100)	Optional contact email
address	VARCHAR(150)	Residential address

Vehicles

Column	Type / Constraint	Description
vehicle_no	VARCHAR(15), PK	Registration number (natural key)
model_name	VARCHAR(50), NOT NULL	Make and model
category	VARCHAR(30), DEFAULT 'Sedan'	Vehicle category
daily_rate	DECIMAL(10,2), NOT NULL, CHECK > 0	Rental charge per day
status	VARCHAR(20), DEFAULT 'Available', CHECK IN (...)	Available / Rented / Maintenance

Rentals

Column	Type / Constraint	Description
rental_id	INT, PK, AUTO_INCREMENT	Surrogate primary key
customer_id	INT, NOT NULL, FK → Customers	Renting customer
vehicle_no	VARCHAR(15), NOT NULL, FK → Vehicles	Rented vehicle
rental_date	DATE, NOT NULL	Date the rental begins
return_date	DATE, CHECK ≥ rental_date	Date the vehicle is returned
status	VARCHAR(20), DEFAULT 'Active', CHECK IN (...)	Active / Completed

Payments

Column	Type / Constraint	Description
payment_id	INT, PK, AUTO_INCREMENT	Surrogate primary key
rental_id	INT, NOT NULL, UNIQUE, FK → Rentals	Rental being paid for (1:1)
amount	DECIMAL(10,2), NOT NULL, CHECK ≥ 0	Amount paid
payment_date	DATE	Date payment was recorded
payment_method	VARCHAR(20), DEFAULT 'Cash', CHECK IN (...)	Cash / Card / Online

2.3 SQL Creation Script

Table structure and constraints are created separately (CREATE TABLE, then ALTER TABLE), which avoids inline CHECK/FOREIGN KEY syntax conflicts across different MySQL/MariaDB versions.

```

CREATE DATABASE car_rental_db;
USE car_rental_db;

CREATE TABLE customers (
  customer_id INT AUTO_INCREMENT PRIMARY KEY,
  customer_name VARCHAR(100) NOT NULL,
  phone VARCHAR(15) NOT NULL,
  email VARCHAR(100),
  address VARCHAR(150)
) ENGINE=InnoDB;

CREATE TABLE vehicles (
  vehicle_no VARCHAR(15) PRIMARY KEY,
  model_name VARCHAR(50) NOT NULL,
  category VARCHAR(30) DEFAULT 'Sedan',
  daily_rate DECIMAL(10,2) NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'Available'
) ENGINE=InnoDB;

CREATE TABLE rentals (
  rental_id INT AUTO_INCREMENT PRIMARY KEY,
  customer_id INT NOT NULL,
  vehicle_no VARCHAR(15) NOT NULL,
  rental_date DATE NOT NULL,
  return_date DATE,
  status VARCHAR(20) NOT NULL DEFAULT 'Active'
) ENGINE=InnoDB;

CREATE TABLE payments (
  payment_id INT AUTO_INCREMENT PRIMARY KEY,
  rental_id INT NOT NULL,
  amount DECIMAL(10,2) NOT NULL,
  payment_date DATE,
  payment_method VARCHAR(20) NOT NULL DEFAULT 'Cash'

```

```

) ENGINE=InnoDB;

ALTER TABLE customers ADD CONSTRAINT uq_customer_phone UNIQUE (phone);
ALTER TABLE payments ADD CONSTRAINT uq_payment_rental UNIQUE (rental_id);
ALTER TABLE rentals ADD CONSTRAINT fk_rentals_customer
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id);
ALTER TABLE rentals ADD CONSTRAINT fk_rentals_vehicle
    FOREIGN KEY (vehicle_no) REFERENCES vehicles(vehicle_no);
ALTER TABLE payments ADD CONSTRAINT fk_payments_rental
    FOREIGN KEY (rental_id) REFERENCES rentals(rental_id);
ALTER TABLE vehicles ADD CONSTRAINT chk_daily_rate CHECK (daily_rate > 0);
ALTER TABLE vehicles ADD CONSTRAINT chk_vehicle_status
    CHECK (status IN ('Available', 'Rented', 'Maintenance'));
ALTER TABLE rentals ADD CONSTRAINT chk_rental_status
    CHECK (status IN ('Active', 'Completed'));
ALTER TABLE rentals ADD CONSTRAINT chk_return_date
    CHECK (return_date IS NULL OR return_date >= rental_date);
ALTER TABLE payments ADD CONSTRAINT chk_amount CHECK (amount >= 0);
ALTER TABLE payments ADD CONSTRAINT chk_payment_method
    CHECK (payment_method IN ('Cash', 'Card', 'Online'));

```

2.4 Sample Data

```

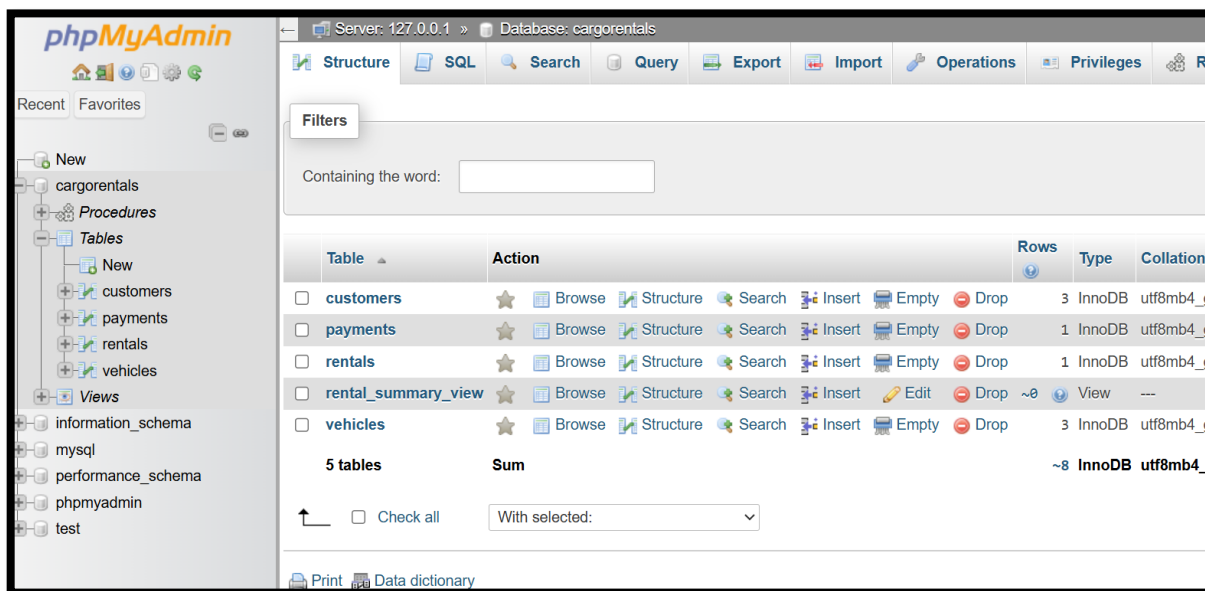
INSERT INTO customers (customer_name, phone, email, address) VALUES
('Ali Khan', '0300-1234567', 'ali.khan@mail.com', 'Muzaffarabad'),
('Ahmed Raza', '0321-2345678', 'ahmed.raza@mail.com', 'Rawalakot'),
('Sara Bilal', '0333-3456789', 'sara.bilal@mail.com', 'Mirpur');

```

```

INSERT INTO vehicles (vehicle_no, model_name, category, daily_rate, status) VALUES
('ABC-123', 'Toyota Corolla', 'Sedan', 5000.00, 'Available'),
('XYZ-789', 'Honda Civic', 'Sedan', 5500.00, 'Available'),
('LHR-456', 'Suzuki Alto', 'Hatchback', 3500.00, 'Available');

```



3. Task 2 — Normalization (1NF, 2NF, 3NF)

The company's original record structure stores every rental as a single flat row:

RentalID	CustomerName	CustomerPhone	VehicleNumber	VehicleModel	DailyRate	RentalDate	ReturnDate	PaymentAmount
R001	Ali Khan	0300-1234567	ABC-123	Toyota Corolla	5000	2026-09-01	2026-09-04	15000

3.1 Anomalies in the Original Structure

- Insertion anomaly: a new vehicle or customer cannot be stored until it is actually part of a rental, since their details only exist inside a rental row.
- Update anomaly: if a customer's phone number changes, every historical rental row for that customer must be found and updated individually.
- Deletion anomaly: deleting a customer's only rental permanently erases that customer's name and phone number, since they are not stored anywhere else.

3.2 First Normal Form (1NF)

Every field already holds a single atomic value with no repeating groups, so the table meets the structural requirement of 1NF. The real problem is that it mixes three different subjects — customer, vehicle, and rental — in one row, which 2NF and 3NF resolve.

3.3 Second Normal Form (2NF)

2NF concerns partial dependency on a composite key. Since the key here is the single attribute RentalID, no partial dependency can exist, so the table is already in 2NF; the transitive dependencies are addressed next.

3.4 Third Normal Form (3NF)

CustomerPhone depends on CustomerName rather than RentalID, and VehicleModel/DailyRate depend on VehicleNumber rather than RentalID — these are transitive dependencies. Removing them gives three relations:

```
CUSTOMER (CustomerPhone [PK], CustomerName)
VEHICLE  (VehicleNumber [PK], VehicleModel, DailyRate)
RENTAL   (RentalID [PK], CustomerPhone [FK], VehicleNumber [FK],
          RentalDate, ReturnDate, PaymentAmount)
```

This is exactly the structure implemented in Section 2, using surrogate/natural keys (customer_id, vehicle_no) and a separate payments table for PaymentAmount.

3.5 Lossless Join and Dependency Preservation

- Lossless join: joining the three relations back together on their foreign keys reconstructs the original flat table exactly, with no missing or extra rows.
- Dependency preservation: every functional dependency in the original table can be checked within one of the three decomposed relations without needing to rejoin them.

4. Task 3 — JOIN Queries

4.1 Query 1 — Basic Rental Details (INNER JOIN)

```
SELECT c.customer_name, v.vehicle_no, v.model_name,
       r.rental_date, r.return_date
FROM rentals r
JOIN customers c ON r.customer_id = c.customer_id
JOIN vehicles v ON r.vehicle_no = v.vehicle_no;
```

The screenshot shows the phpMyAdmin interface with the 'cargorentals' database selected. The 'SQL' tab is active, displaying a query that successfully executed. The query result shows one row of data.

Server: 127.0.0.1 » Database: cargorentals

Showing rows 0 - 0 (1 total, Query took 0.0004 seconds.)

```
SELECT c.CustomerName, v.VehicleNumber, v.VehicleModel, r.RentalDate, r.ReturnDate FROM Rentals r JOIN Customers c ON c.CustomerID = r.CustomerID JOIN Vehicles v ON v.VehicleID = r.VehicleID;
```

Query results operations: Print, Copy to clipboard, Export, Display chart, Create view

CustomerName	VehicleNumber	VehicleModel	RentalDate	ReturnDate
Ali Khan	ABC-123	Toyota Corolla	2026-09-01	2026-09-04

4.2 Query 2 — All Customers and Their Rented Vehicles (LEFT JOIN)

```
SELECT c.customer_name, v.vehicle_no, v.model_name
FROM customers c
LEFT JOIN rentals r ON c.customer_id = r.customer_id
LEFT JOIN vehicles v ON r.vehicle_no = v.vehicle_no;
```

The screenshot shows the phpMyAdmin interface with the 'cargorentals' database selected. The 'SQL' tab is active, displaying a query that successfully executed. The query result shows three rows of data, including customers with no rentals.

Server: 127.0.0.1 » Database: cargorentals

Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete features are not available.

Showing rows 0 - 2 (3 total, Query took 0.0004 seconds.)

```
SELECT c.CustomerName, c.CustomerPhone, v.VehicleNumber, v.VehicleModel FROM Customers c LEFT JOIN Rentals r ON c.CustomerID = r.CustomerID LEFT JOIN Vehicles v ON r.VehicleID = v.VehicleID;
```

Query results operations: Print, Copy to clipboard, Export, Display chart, Create view

CustomerName	CustomerPhone	VehicleNumber	VehicleModel
Ali Khan	0300-1234567	ABC-123	Toyota Corolla
Sara Ahmed	0321-9876543	NULL	NULL
Usman Tariq	0333-5554433	NULL	NULL

4.3 Query 3 — All Vehicles and Current Rental Info (LEFT JOIN)

```
SELECT v.vehicle_no, v.model_name, v.status, r.rental_id
FROM vehicles v
LEFT JOIN rentals r ON v.vehicle_no = r.vehicle_no AND r.status = 'Active';
```

The screenshot shows the phpMyAdmin interface with the 'cargorentals' database selected. The 'SQL' tab is active, displaying a query that performs a LEFT JOIN between the 'Vehicles' and 'Rentals' tables. The query filters for vehicles with a status of 'Rented'. The result set shows three rows, with the first two containing data and the third being a NULL result from the left join.

VehicleNumber	VehicleModel	VehicleStatus	RentalDate	ReturnDate
ABC-123	Toyota Corolla	Rented	2026-09-01	2026-09-04
XYZ-789	Honda Civic	Available	NULL	NULL
LMN-456	Suzuki Swift	Available	NULL	NULL

4.4 Query 4 — Total Rentals per Customer (LEFT JOIN + GROUP BY)

```
SELECT c.customer_name, COUNT(r.rental_id) AS total_rentals
FROM customers c
LEFT JOIN rentals r ON c.customer_id = r.customer_id
GROUP BY c.customer_id, c.customer_name;
```

The screenshot shows the phpMyAdmin interface with the 'cargorentals' database selected. The 'SQL' tab is active, displaying a query that performs a LEFT JOIN between the 'Customers' and 'Rentals' tables, grouped by customer. The result set shows three rows, each representing a customer and their total number of rentals.

CustomerName	TotalRentals
Ali Khan	1
Sara Ahmed	0
Usman Tariq	0

5. Task 4 — Database View

A single view, `rental_summary_view`, consolidates customer, vehicle, rental, and payment data into one queryable report.

```
CREATE VIEW rental_summary_view AS
SELECT r.rental_id, c.customer_name, c.phone,
       v.vehicle_no, v.model_name,
       r.rental_date, r.return_date, r.status AS rental_status,
       p.amount, p.payment_method
FROM rentals r
```

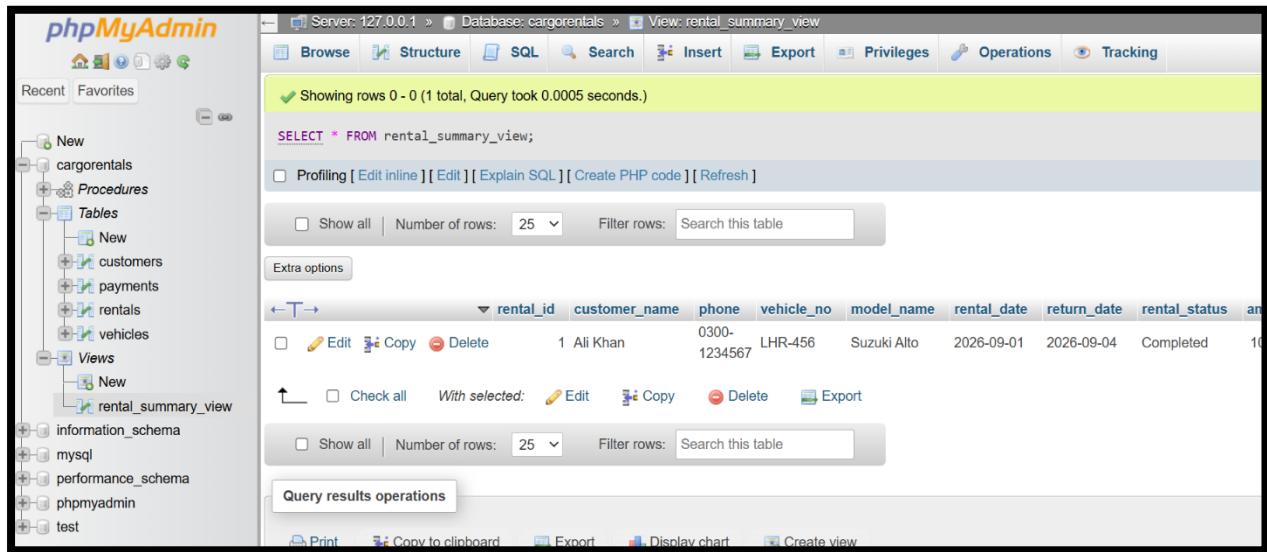
```

JOIN customers c ON r.customer_id = c.customer_id
JOIN vehicles v ON r.vehicle_no = v.vehicle_no
LEFT JOIN payments p ON r.rental_id = p.rental_id;

SELECT * FROM rental_summary_view;

```

This view saves staff from re-writing the same four-table join every time a rental report is needed, lets reporting tools be given read-only access without exposing the base tables directly, and keeps the join logic in one place if the schema changes later.



6. Tasks 5 & 6 — Triggers and Stored Procedure

6.1 Triggers

Three triggers keep vehicles.status accurate automatically, which is what enforces the rule that a vehicle cannot be rented twice at the same time — the database rejects the attempt instead of relying on the application to check dates.

```

DELIMITER $$

CREATE TRIGGER prevent_double_booking
BEFORE INSERT ON rentals
FOR EACH ROW
BEGIN
    DECLARE v_status VARCHAR(20);
    SELECT status INTO v_status FROM vehicles WHERE vehicle_no = NEW.vehicle_no;
    IF v_status <> 'Available' THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'This vehicle is already rented.';
    END IF;
END$$

CREATE TRIGGER update_car_status
AFTER INSERT ON rentals
FOR EACH ROW
BEGIN
    UPDATE vehicles SET status = 'Rented' WHERE vehicle_no = NEW.vehicle_no;
END$$

CREATE TRIGGER release_car_on_return
AFTER UPDATE ON rentals
FOR EACH ROW
BEGIN
    IF NEW.status = 'Completed' AND OLD.status = 'Active' THEN
        UPDATE vehicles SET status = 'Available' WHERE vehicle_no = NEW.vehicle_no;
    END IF;
END$$

```

```
END$$
```

```
DELIMITER ;
```

Without these triggers, checking whether a vehicle is currently rented would require re-running a date-range query against the entire rentals table on every lookup. Storing and automatically maintaining status on the vehicles row itself turns that check into a single indexed column read, which is both faster and guarantees the value can never drift out of sync with the actual rental records.

```
SELECT vehicle_no, model_name, status FROM vehicles WHERE vehicle_no = 'ABC-123';
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

```
UPDATE `vehicles` SET `status` = 'rented' WHERE `vehicles`.`vehicle_no` = 'ABC-123';
```

[Edit inline](#) [[Edit](#)] [[Create PHP code](#)]

☐ Show all | Number of rows: 25 ▾ Filter rows:

Extra options



vehicle_no

model_name

status



Edit



Copy



Delete

ABC-123

Toyota Corolla

rented

Error

SQL query: [Copy](#)

```
INSERT INTO rentals (customer_id, vehicle_no, rental_date, status)
VALUES (2, 'ABC-123', '2026-09-10', 'Active');
```

MySQL said: 

#1644 - This vehicle is already rented.

6.2 Stored Procedure — addNewRental

This procedure registers a new rental and calculates the total charge from the rental period and the vehicle's daily rate in one call.

```
DELIMITER $$
```

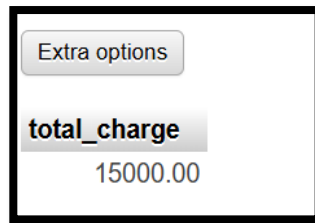
```

CREATE PROCEDURE addNewRental (
  IN p_customer_id INT,
  IN p_vehicle_no  VARCHAR(15),
  IN p_rental_date DATE,
  IN p_return_date DATE,
  OUT p_total_charge DECIMAL(10,2)
)
BEGIN
  DECLARE v_rate DECIMAL(10,2);
  DECLARE v_days INT;
  SELECT daily_rate INTO v_rate FROM vehicles WHERE vehicle_no = p_vehicle_no;
  SET v_days = DATEDIFF(p_return_date, p_rental_date);
  SET p_total_charge = v_days * v_rate;
  INSERT INTO rentals (customer_id, vehicle_no, rental_date, return_date, status)
  VALUES (p_customer_id, p_vehicle_no, p_rental_date, p_return_date, 'Active');
END$$

DELIMITER ;

-- Example calls:
CALL addNewRental(1, 'ABC-123', '2026-09-01', '2026-09-04', @charge1);
SELECT @charge1 AS total_charge;

```



7. Task 7 — Optimization Analysis

7.1 Identified Inefficiency

The rentals table's foreign key columns, `customer_id` and `vehicle_no`, have no index beyond the automatic primary key index. Every JOIN query in Section 4, the view, and the availability check inside `addNewRental` all filter or join on one of these columns.

7.2 Performance Impact

Without an index, the engine must scan every row of rentals to satisfy a WHERE or JOIN condition on these columns — negligible with three rows, but it scales linearly with rental volume and would slow down every report and every rental booking as the table grows.

7.3 Proposed Improvement

```

EXPLAIN SELECT * FROM rentals WHERE customer_id = 2;

CREATE INDEX idx_rentals_customer ON rentals (customer_id);
CREATE INDEX idx_rentals_vehicle  ON rentals (vehicle_no);

EXPLAIN SELECT * FROM rentals WHERE customer_id = 2;

```

Adding a B-Tree index on each foreign key column lets the engine locate matching rows directly instead of scanning the whole table, changing the query plan's access type from a full table scan to an indexed lookup — visible in the EXPLAIN output before and after the index is created.

Before

Your SQL query has been executed successfully.

```
EXPLAIN SELECT * FROM rentals WHERE customer_id = 2;
```

[[Edit inline](#)] [[Edit](#)] [[Skip Explain SQL](#)] [[Create PHP code](#)]

Extra options

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	rentals	ref	fk_rentals_customer	fk_rentals_customer	4	const	1	

After

Your SQL query has been executed successfully.

```
EXPLAIN SELECT * FROM rentals WHERE customer_id = 2;
```

[[Edit inline](#)] [[Edit](#)] [[Skip Explain SQL](#)] [[Create PHP code](#)]

Extra options

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	rentals	ref	idx_rentals_customer	idx_rentals_customer	4	const	1	

8. Conclusion

This report has taken CarGo Rentals from an inconsistent, spreadsheet-based record of rentals to a fully relational database solution. Normalizing the original flat structure eliminated the redundancy and anomalies identified in Section 3, while primary keys, foreign keys, and CHECK/UNIQUE/DEFAULT constraints now enforce data integrity at the database level. The JOIN queries answer every required reporting question, the view consolidates that reporting into a single reusable object, and the triggers together with addNewRental automate and enforce the rule that a vehicle cannot be double-booked. Finally, the indexing strategy in Section 7 ensures the design will continue to perform well as transaction volume grows.